The Sound-Maze Design Book

Redesigning Pac-Man-like maze chase games for blind and visually impaired players

Prepared for: A design and engineering team building a blind-first, keyboard-and-speaker maze chase game.
Purpose: Teach the science behind maze chase games and translate it into a playable non-visual system.
Legal note: This book uses Pac-Man as a genre reference. Build an original game, original characters, original level layouts, original sounds, and original branding.

Version 1.0 - June 2026

Table of Contents

Core thesis: A blind-first maze chase game is not a visual game with audio added. It is a real-time decision system where sound, rhythm, memory, and route planning replace sight.

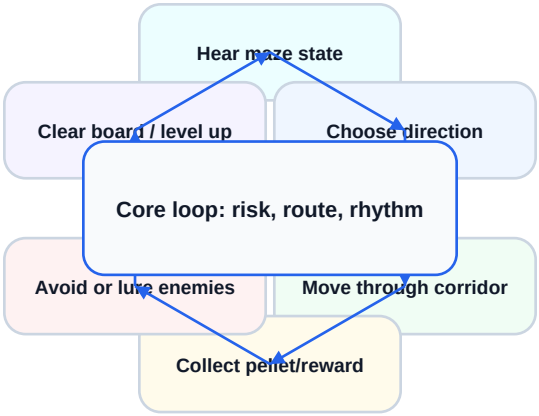
1. Executive Design Doctrine

1.1 What we are building

The target product is a **blind-first maze chase game** playable with only keyboard input and speaker or headphone sound. The player travels through a maze, collects rewards, avoids enemies, uses temporary power states, and clears increasingly complex levels. The famous arcade maze-chase formula is the inspiration, but the design must be original in art, audio, map layout, enemy characters, names, and story.

The design goal is not pity accessibility. The goal is a game that feels fast, skillful, learnable, tense, and replayable for visually impaired players. Sighted players should be able to enjoy it too, but the primary experience must work without vision.

The maze chase core loop



The game is a repeating loop of hearing state, choosing movement, collecting reward, managing enemies, and clearing the board.

1.2 The five laws of blind-first game design

- **Law 1 - Sound is gameplay.** Audio is not decoration. It is how players perceive space, danger, reward, timing, and progress.

- **Law 2 - Every sound must answer a question.** Where am I? What is near me? What changed? What can I do next? What is dangerous?

- **Law 3 - Reduce hidden state.** If the player dies, they should know why. If danger rises, they should hear it before impact.

- **Law 4 - Build mental maps.** The player should learn rooms, routes, landmarks, junctions, enemy patterns, and safe pockets.

- **Law 5 - Difficulty changes one variable at a time.** Increase speed, enemies, branching, silence, or timing pressure gradually, not all at once.

1.3 What not to do

- Do not simply read the entire screen through text-to-speech during action. Real-time games need short cues and rhythm, not paragraphs.

- Do not use one generic beep for everything. A beep-only game becomes cognitive noise.

- Do not copy the original arcade maze. A blind-first game needs routes designed around audio landmarks and learnable junctions.

- Do not make the trainer view the real product. The blind player experience must remain complete without the trainer screen.

- Do not validate the game only with sighted testers. Blind and low-vision players must be included before release.

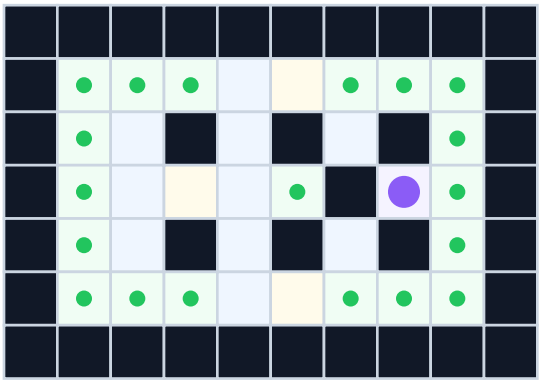
2. The Science of Maze Chase Games

2.1 The genre formula

A maze chase game is built from a small set of rules that create a large skill space: a fixed or semi-fixed maze, collectible items, pursuing enemies, temporary power reversals, scoring, lives, and level progression. Pac-Man popularized this structure with dots, energizers, ghosts, fruit, warp tunnels, and increasing difficulty. In the original design tradition, enemies are not only obstacles; their different movement behaviors create readable personalities.

For blind-first redesign, the formula remains useful because it has clear spatial rules. The challenge is converting visual spatial information into a compressed audio language that does not overload the player.

Tile grammar of a sound maze



A non-visual maze is a grammar of meaningful tiles, not a picture.

W	Wall
C	Corridor
J	Junction
P	Pellet
E	Enemy
S	Safe pocket

A maze must be represented as meaningful tile types that can be described through sound and logic.

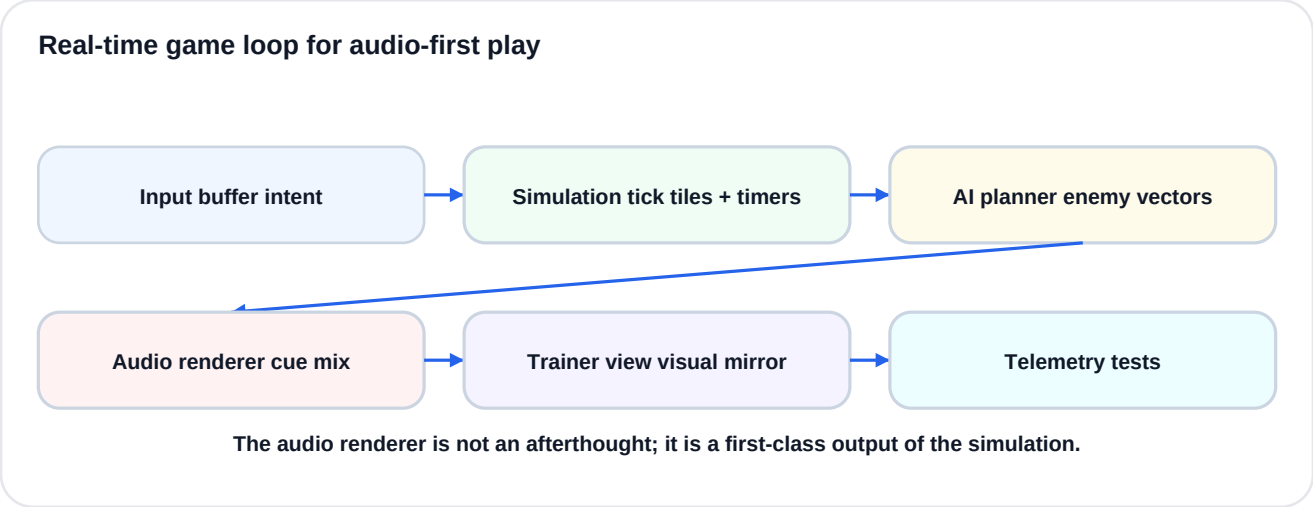
2.2 Why the classic formula works

- **It has a simple verb set:** move, collect, avoid, lure, reverse, clear.

- **It has local decisions:** at each junction, the player chooses a direction based on reward and danger.
- **It has global strategy:** the player plans routes, saves power items, and manages enemy locations.
- **It has tension waves:** chase pressure rises, power mode flips the emotion, then danger returns.
- **It has mastery:** repeated levels let players learn patterns and improve.

2.3 The three clocks

A maze chase game is controlled by three clocks. The **movement clock** determines how quickly player and enemies move. The **decision clock** determines how often the player reaches junctions or must react. The **mode clock** determines when enemy behaviors change, power states begin, and power states expire. Blind-first design must make all three clocks audible.



Input, simulation, AI, audio rendering, trainer visualization, and telemetry should be separate but synchronized.

2.4 Risk and reward

The emotional engine is risk versus reward. Pellets create a safe micro-reward. Bonus items create route temptation. Enemies create pressure. Power items create a temporary reversal where the hunted player becomes the hunter. A good level repeatedly asks: should I take the safe route, or risk the better route?

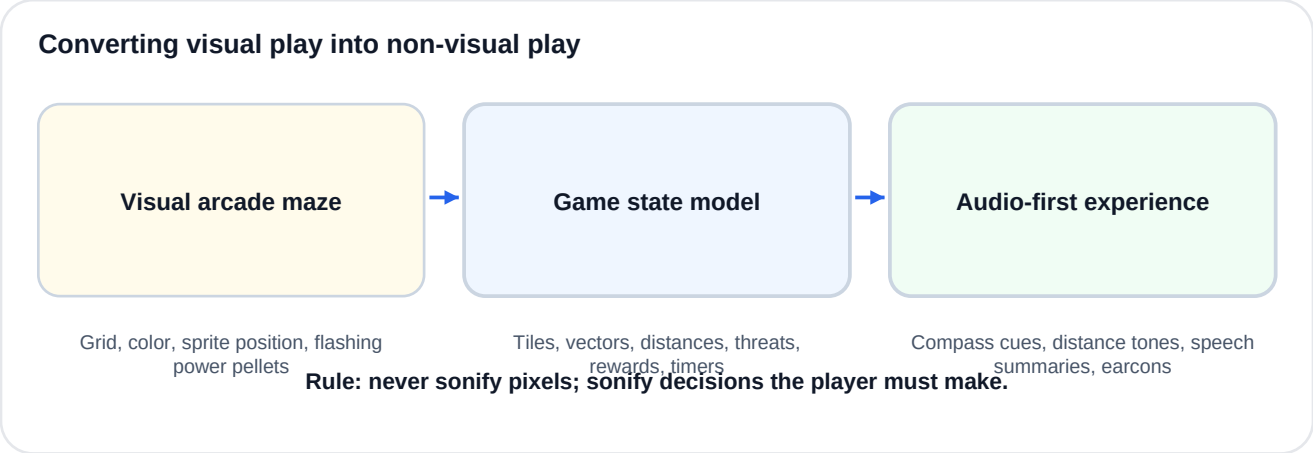
Mechanic	Player question	Blind-first audio expression
Pellets	Am I making progress?	Soft rhythmic ticks along the current route; pitch rises as a row is cleared.
Junction	Which ways are open?	Directional cues or a short scan: "left fruit, right danger, up pellets."
Enemy	How urgent is danger?	Panned movement cue with intensity based on time-to-collision.
Power item	Can I reverse the chase?	Distinct bass-to-bright transformation plus countdown warning.
Level clear	Did I finish?	Strong musical cadence, spoken score, next-level preview.

3. Blind and Low-Vision Interaction Model

3.1 Design around ability, not deficit

Blind and low-vision players are not one group with one ability profile. Some players are fully blind from birth and may build strong auditory and tactile mental maps. Some have low vision and may use enlarged, high-contrast visuals. Some use screen readers daily; others rely more on memory, keyboard habits, or mobile voice interfaces. The game should support multiple modes while keeping the core playable by sound alone.

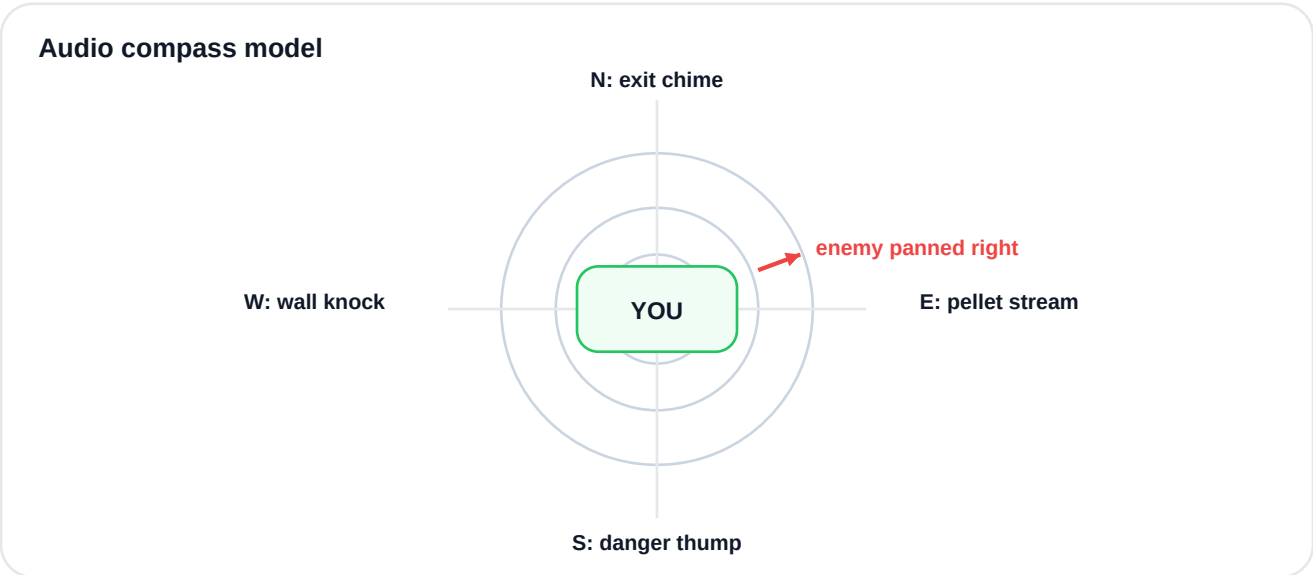
The most important design shift is this: sighted players glance; blind players query, listen, infer, remember, and act. Therefore the game must give immediate action cues, optional deeper scan cues, and stable landmarks that support memory.



The game should transform raw visual state into player decisions, then into compact audio.

3.2 The mental map

The player must build a mental model of the maze. The mental map should not require memorizing every tile at once. It should emerge from zones, landmarks, repeatable route names, audio textures, and junction summaries. A level should have a small number of memorable places: start, safe corridor, enemy gate, power corner, fruit route, loop, shortcut, and danger pocket.



Spatial audio should behave like a compass: left/right panning, distance intensity, and distinct cue families.

3.3 Cognitive load budget

The most common failure in audio-first action games is over-speaking. Spoken audio is slow; action games are fast. Reserve speech for menus, tutorials, post-death explanations, and optional scan mode. During live action, use short

earcons, auditory icons, panning, pitch, rhythm, and silence.

The audio mix should prioritize survival cues, then navigation, then reward, then ambience.

3.4 Accessibility needs to support

- Keyboard-only operation with remappable keys.
- Screen-reader-compatible menus, but not screen-reader-dependent live gameplay.
- Adjustable cue density: beginner, standard, expert.
- Mono-compatible fallback for players without headphones.
- Reduced-stress mode: slower enemies, longer warnings, no sudden loud sounds.
- Replayable tutorial with no penalty.
- Trainer/supervisor visual mirror for teaching and debugging.

4. Transforming a Visual Maze into a Sound Maze

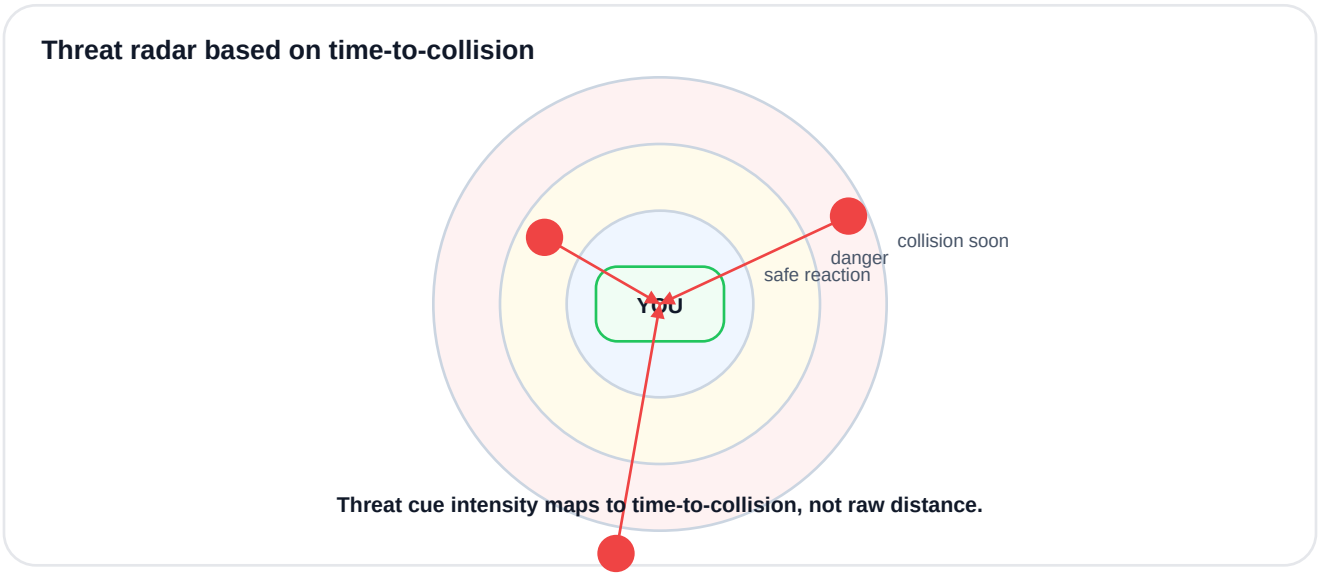
4.1 The sound maze is not a picture

A visual maze can show everything at once. A sound maze must reveal information over time. This means the level designer must decide what the player needs now, what they can request, and what they can learn through repeated play.

Design principle: sonify decisions, not decorations. If a sound does not change a decision, reduce it, delay it, or remove it.

4.2 Three information ranges

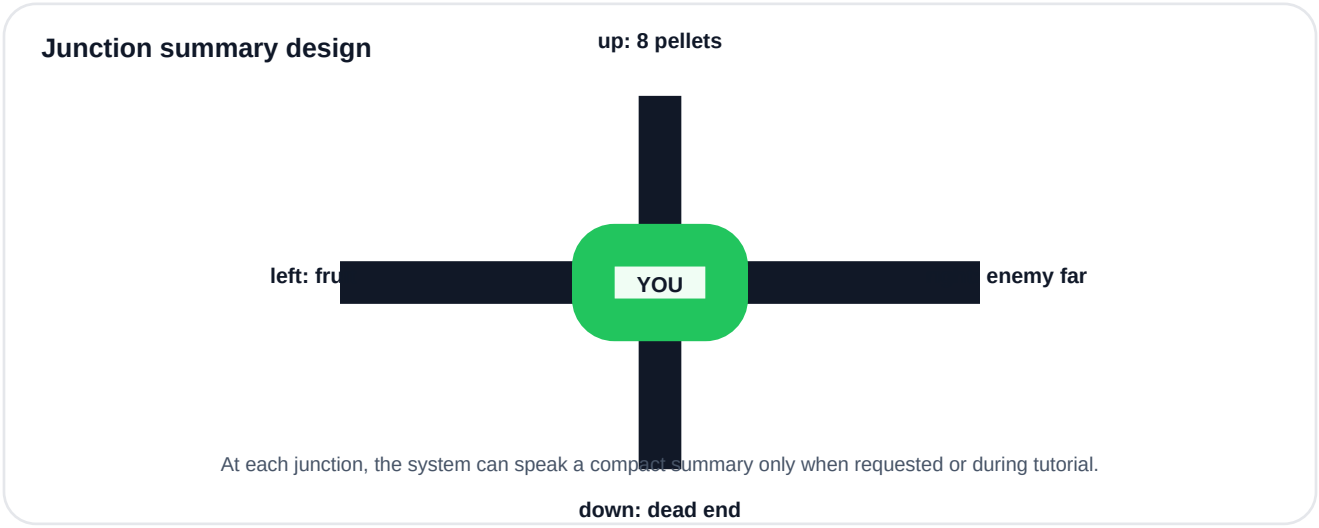
Range	Purpose	Example cues
Immediate tile	Confirm movement and collision.	Footstep/tick per tile, wall bump, pellet tick.
Near field	Support reaction and junction choices.	Enemy panning, open-exit chime, danger pulse.
Far field	Support planning.	Requested scan: fruit north, power west, enemy gate center.



Danger should be calculated from time-to-collision and route topology, not just raw Euclidean distance.

4.3 The scan mechanic

The scan mechanic is the blind player equivalent of a quick glance. Pressing a scan key should not pause the entire game unless the mode is beginner. Instead, it can deliver a short directional summary: "left clear, up pellets, right enemy, down wall." Advanced players can reduce or disable scan speech and rely on directional audio.



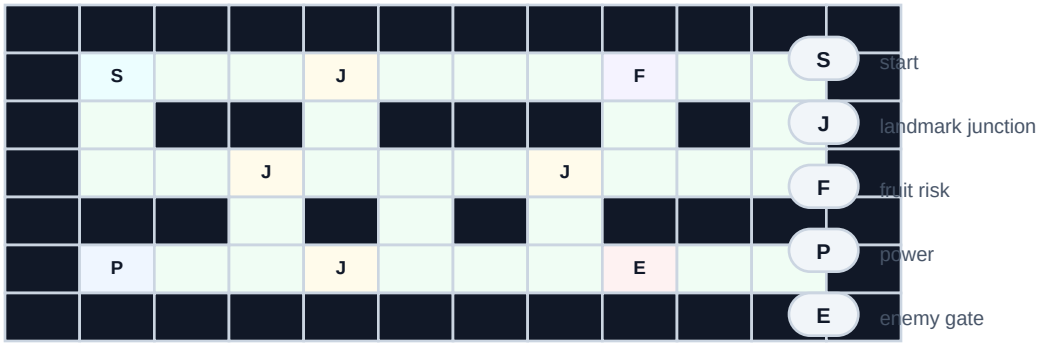
At decision points, the system can expose open routes and meaningful nearby objects.

4.4 Landmark audio

A level should have landmarks that sound different. For example, the fruit corridor could have a soft marimba ambience, the enemy gate could have a low machine hum, and the safe loop could have a light wind texture. Landmarks should be subtle and always lower priority than danger cues.

- Use a unique ambience per zone, not per tile.
- Use repeated motifs so players can learn them.
- Avoid continuous clutter; let empty corridors be calm.
- Keep all landmark sounds optional in accessibility settings.

Training level blueprint



Training maps use landmark junctions before they use complex loops.

A level blueprint should label start, junctions, power, fruit, enemy gate, and safe routes.

5. Keyboard-Only Control Design

5.1 The verb set

The game should be controllable through a small number of verbs: move up, move down, move left, move right, scan surroundings, repeat last cue, pause, confirm, back. The player should never need a mouse. Every menu action must have a keyboard path.

Keyboard and six-key control modes

Design choice: all modes expose the same game verbs: move, scan, repeat, pause, confirm, back.

The same game verbs can be mapped to arrow keys, WASD, or the six-key mode from the earlier concept.

5.2 Direction queuing

Classic maze chase games often allow the player to press a turn direction just before reaching a corner; the game executes it when possible. This is essential for blind-first play. The player cannot visually time every corner, so the system should support forgiving direction queuing, turn previews, and wall feedback.

- **Direction buffer:** remember the last valid direction intent for 250-500 ms or until the next junction.

- **Turn preview:** when a turn is queued, play a tiny directional click so the player knows it was accepted.

- **Wall rejection:** if a direction is impossible, play a short wall knock and keep the current direction.

- **Beginner assist:** allow automatic lane-centering and slightly longer turn windows.

5.3 Six-key mode

From your earlier design direction, the game should also support a six-key layout for players who prefer a compact keyboard zone or a future tactile controller. A recommended mapping is S, D, F, J, K, L: left, up, right, scan, confirm/repeat, back/pause. Down can be a chord such as D+K or assigned to J depending on user testing. A safer first version is six physical keys plus a mode switch where left/up/right/down are mapped to four of the six keys and scan/repeat use the remaining two.

Mode	Keys	Why it exists
Arrow mode	Arrow keys + Space scan + Esc pause	Fast for sighted and many keyboard users.
WASD mode	WASD + Q scan + R repeat	Common PC gaming layout.
Six-key mode	S D F J K L	Supports compact tactile keyboard experiments and one-hand training.
Trainer hotkeys	1-9 overlays, F1 help	Lets a trainer inspect without interfering with player controls.

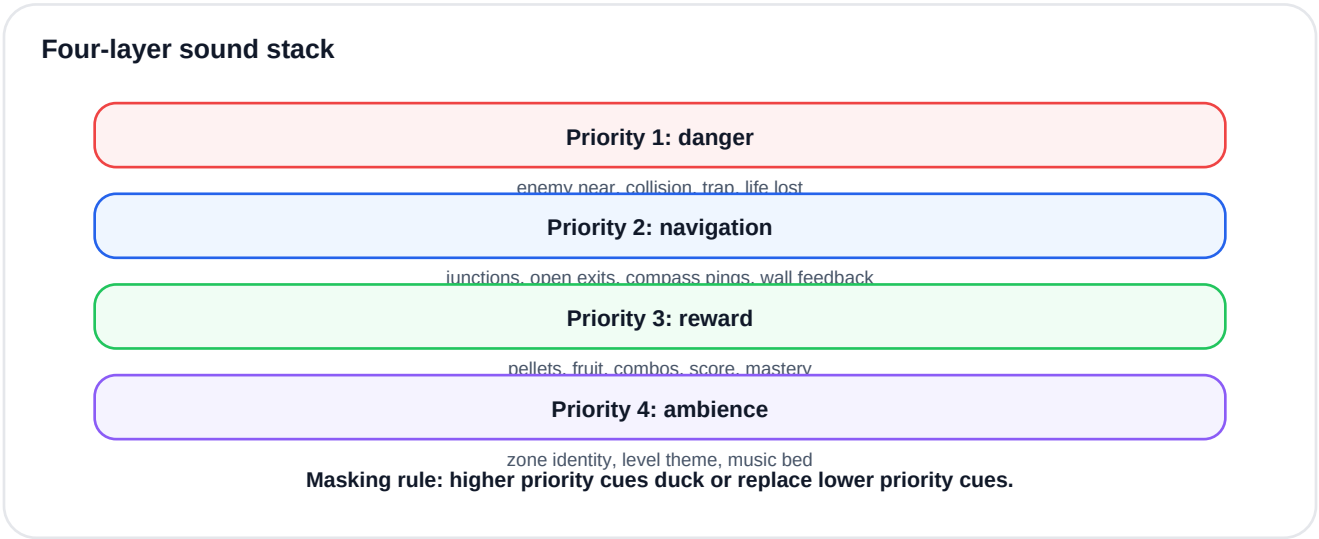
5.4 Error recovery

A blind player should be able to recover from confusion without quitting the game. Provide a panic key that gives a short safe-state summary or slows the game for two seconds in training mode. This is not cheating; it is equivalent to visual reorientation.

6. Audio UX and Sound Grammar

6.1 Cue families

A good audio-first game uses different cue families for different kinds of information. The player should never need to ask, "Was that reward, danger, or navigation?" Each family should occupy a different timbre, pitch range, rhythm, and spatial behavior.



Danger, navigation, reward, and ambience should be mixed as separate layers with explicit priority.

Cue family	Recommended sound language	Example
Walls	Short dry knock, no reverb	Invalid turn: small wooden tap.
Pellets	Tiny bright ticks, rhythmic	Each collected pellet gives a soft tick.
Junctions	Directional chimes	Open exits have short left/up/right tones.
Enemies	Moving tonal signatures	Hunter has low pulse; ambusher has sharp syncopation.
Power	Full-mode transformation	Bass swell, brighter enemy cues, countdown ticks.
Fruit/bonus	Melodic lure	Distinct melody placed in direction of bonus.
Trainer/status	Speech only	"Level 2, score 940, three lives."

6.2 Spatial audio rules

- Use left-right panning for horizontal direction.

- Use pitch or filter brightness for vertical direction if the game has up/down choices on a 2D map.

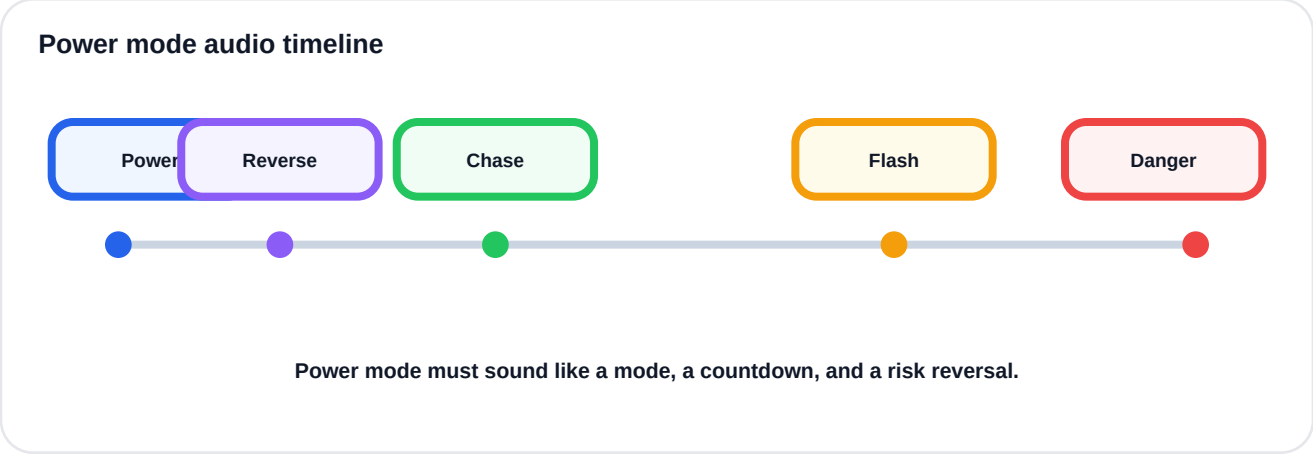
- Use volume and pulse rate for urgency, but cap loudness to avoid fatigue.

- Use stereo headphones as the best experience, but support mono by replacing panning with spoken or tonal direction codes.

- Never rely on a single audio dimension for critical information. Pair direction with timbre or rhythm.

6.3 Speech rules

Speech is powerful but expensive. During gameplay, speech should be short, interruptible, and never block critical danger cues. Good live speech examples: "enemy right," "power west," "dead end," "one life." Bad live speech example: "There are three pellets in the corridor immediately to your left and an enemy approximately six tiles away from your current coordinate."



Power mode should communicate activation, reversal, chase opportunity, warning, and danger return.

6.4 Audio fatigue and user settings

- Master volume, speech volume, effects volume, ambience volume.

- Cue density: rich, standard, minimal.

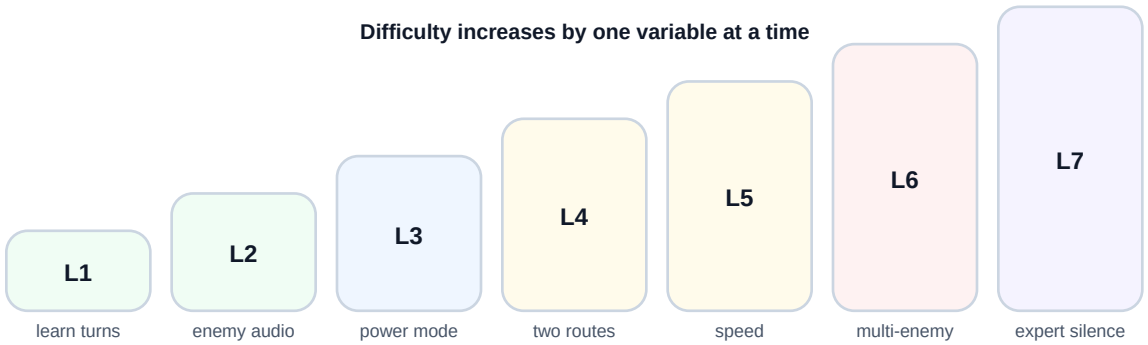
- Threat warning distance: short, medium, long.
- Speech speed and voice selection.
- Mono mode, headphone mode, low-frequency reduction, sudden-sound reduction.
- Practice mode with slow simulation and longer scan summaries.

7. Level Design Science

7.1 The level is a lesson

Every level should teach or test one idea. Level 1 teaches moving and turning. Level 2 teaches enemy direction. Level 3 teaches power reversal. Level 4 teaches route choice. Level 5 teaches planning under speed. Expert levels can reduce guidance, add more enemies, and use subtler landmarks.

Difficulty ladder



Progression should introduce one new pressure variable at a time.

7.2 Maze anatomy

Element	Purpose	Blind-first design rule
Corridor	Movement rhythm	Long enough to hear progress, not so long that nothing happens.
Junction	Decision point	Must have clear directional feedback and readable consequences.
Loop	Escape and strategy	Give players a way to outmaneuver enemies using memory.
Dead end	Risk trap	Introduce only after the player has warning and recovery tools.
Power corner	Risk reversal	Place where getting there under pressure is meaningful.
Safe pocket	Recovery	A small place to reorient, not a permanent shelter.
Warp/shortcut	Advanced routing	Use distinctive transition sound and clear exit cue.

7.3 Level templates

- **Template A - Training loop:** one loop, four junctions, one slow enemy, high cue density.

- **Template B - Crossroads:** central hub, four arms, fruit lure, beginner scan support.

- **Template C - Double loop:** two connected loops, enemy can pressure one loop while player escapes through another.

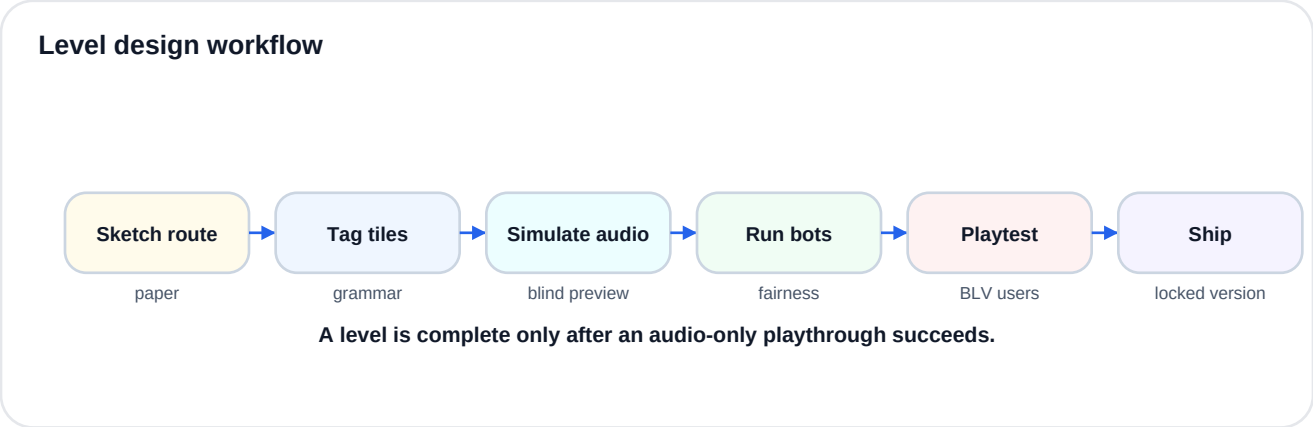
- **Template D - Power route:** power item placed behind a risky corridor, teaching route planning.

- **Template E - Expert maze:** multiple loops, reduced speech, faster enemies, timed bonus routes.

7.4 Designing fairness

A blind-first level is unfair when it requires visual reaction time. It is fair when danger is predictable, audible, and avoidable through skill. Fair does not mean easy. It means the player can understand the loss and improve next time.

Fair challenge	Unfair barrier
Enemy cue grows faster as it approaches.	Enemy appears silently near the player.
Dead end has a warning texture.	Dead end looks different visually but sounds identical.
Power mode has countdown warning.	Power expires with no cue.
Direction queue forgives early input.	Turn input requires exact visual tile timing.
Death replay explains the cause.	Player only hears a generic failure sound.

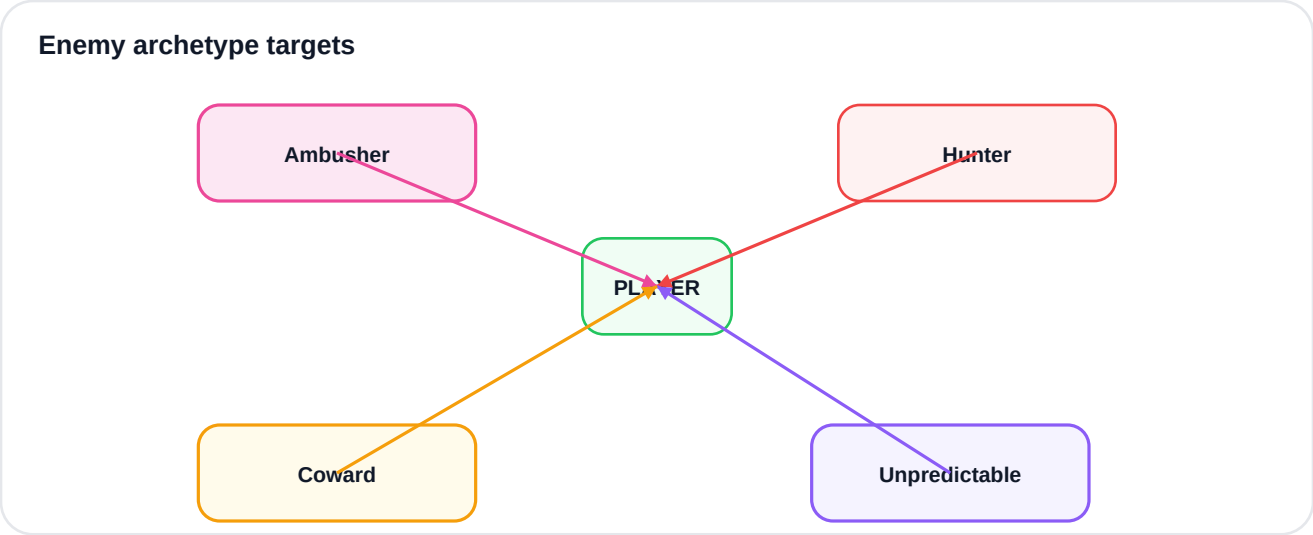


Build, tag, simulate, test, and lock every level through an audio-first pipeline.

8. Enemy AI and Fair Pressure

8.1 Enemy behavior as personality

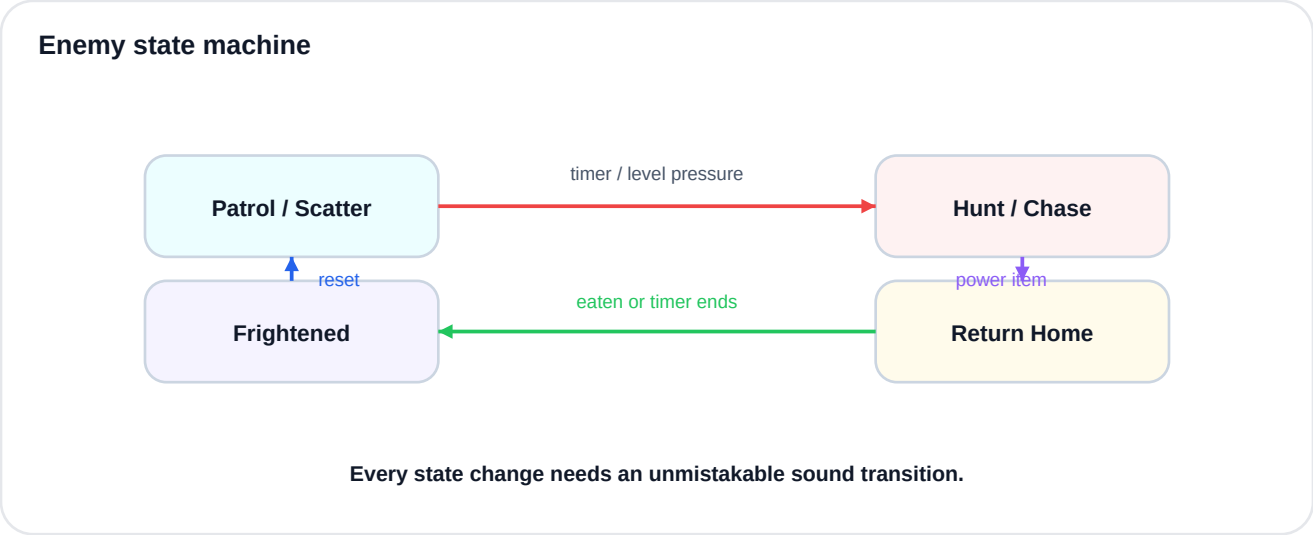
One reason the classic maze chase formula remains interesting is that enemies can have distinct behavior patterns. A direct hunter, an ambusher, an unpredictable vector-blend enemy, and a near/far switch enemy create different pressures without needing complex graphics. For blind-first play, each behavior must also have a distinct audio identity.



Enemy personalities should create different route pressures and different sounds.

8.2 State machine

Enemies should move through clear states: patrol/scatter, hunt/chase, frightened, and return-home. A state change should always have an audio transition. This helps the player understand the game clock and prevents deaths that feel random.



Each state changes target logic, speed, and audio identity.

8.3 Threat calculation

For audio warnings, time-to-collision is more useful than distance. An enemy six tiles away in the same corridor is more dangerous than an enemy four tiles away behind a wall. The engine should calculate route distance, enemy direction, player direction, and speed to assign warning priority.

- **Green threat:** enemy is nearby but not on an intercept route.
- **Amber threat:** enemy can intercept within a few seconds.
- **Red threat:** collision likely unless the player turns, powers up, or escapes.
- **Silent threat:** no cue required because it would distract from more urgent information.

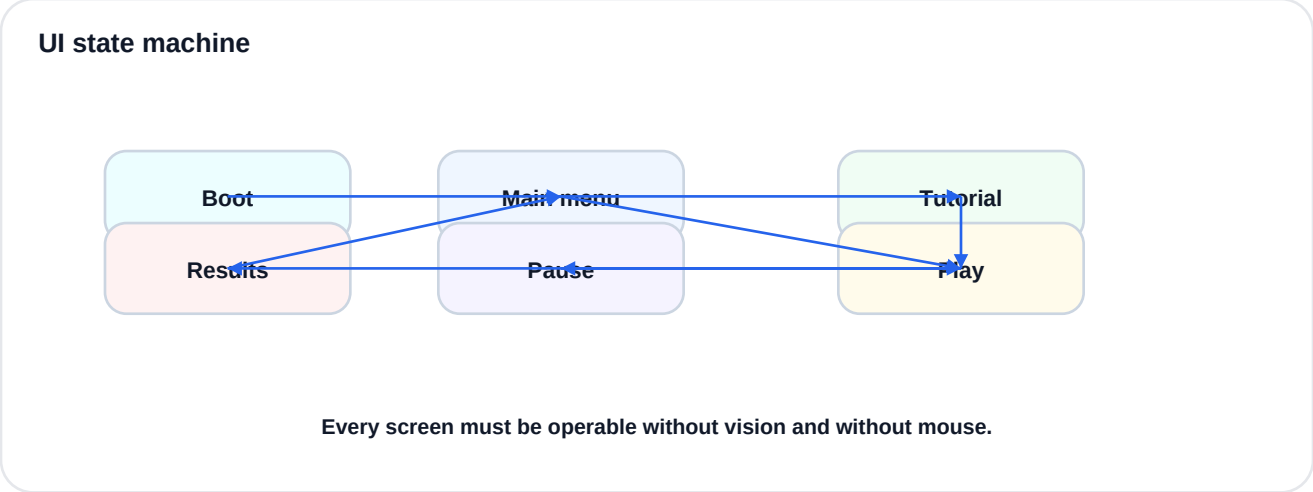
8.4 Avoiding audio overload with multiple enemies

Do not play four full enemy tracks all the time. Mix only the most relevant enemy at full priority, compress the second as a lighter cue, and summarize the rest through scan mode. The trainer view can show all enemies visually, but the player audio should stay decision-focused.

9. Menus, Onboarding, Trainer Interface, and UI/UX

9.1 Menu UX

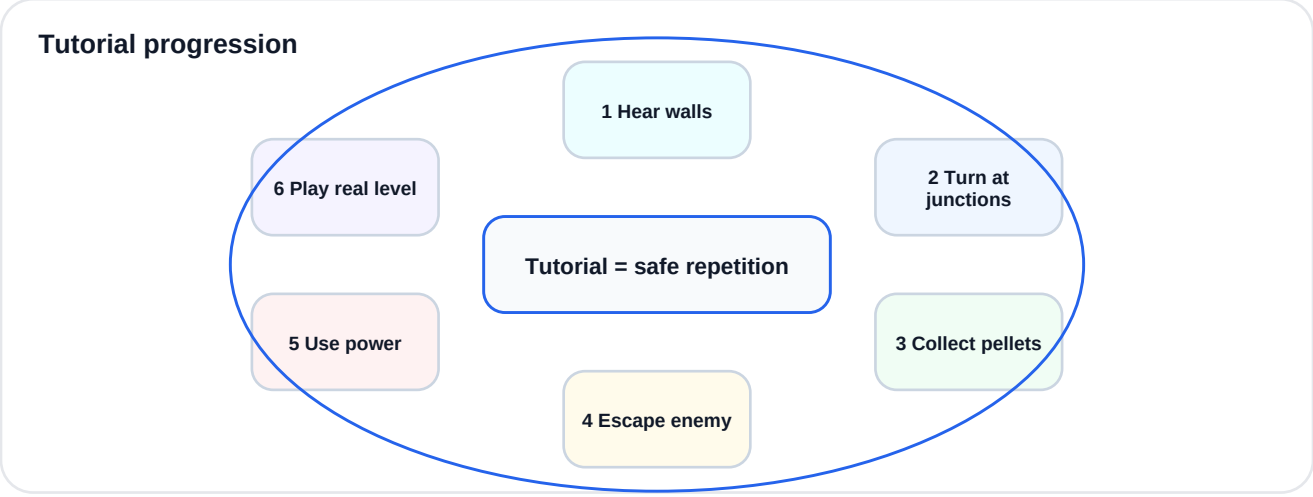
Menus should be screen-reader compatible, keyboard-only, and self-voicing. The first launch should speak: game title, current menu item, how to navigate, and how to start tutorial. Each menu item should respond to arrow keys and six-key navigation. Do not require mouse hover, drag, or visual-only settings.



The UI should have a small set of states and predictable keyboard transitions.

9.2 Onboarding

The tutorial should be the first major design feature, not a final add-on. It should teach sound meanings in a safe environment, then combine them gradually. The player should be able to repeat any tutorial module.

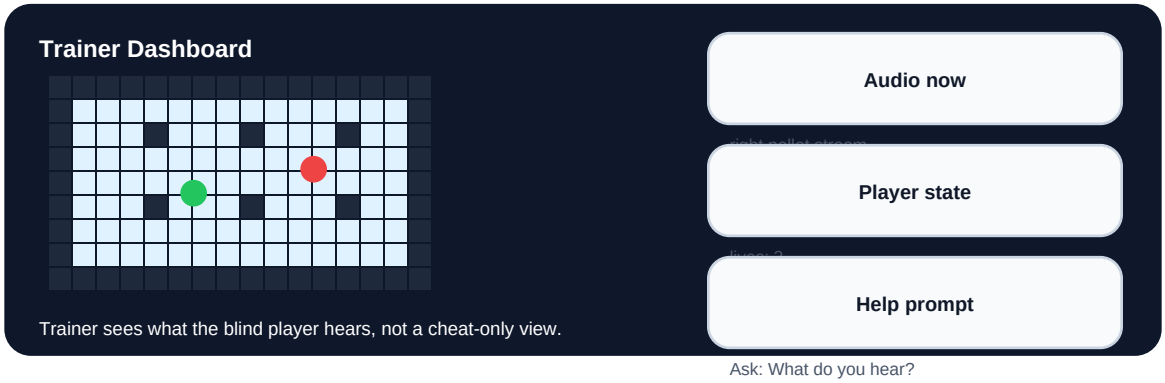


Teach walls, turns, pellets, enemies, power mode, then real play.

9.3 Trainer interface

Because your earlier direction included a trainer/supervisor screen, the game should include a visual dashboard for teachers, researchers, parents, or sighted assistants. This is not required for the blind player to play. It is a mirror for coaching, observation, debugging, and classroom use.

Trainer dashboard mockup



The trainer view mirrors the maze, current audio cues, player state, and coaching prompts.

9.4 Trainer ethics

- The trainer should coach by asking what the player hears before giving the answer.
- The trainer screen should show cue history so the trainer can understand the player experience.
- Avoid turning the trainer into the real player. The blind player must own decisions.
- Use the trainer view for accessibility testing, not as a permanent crutch.

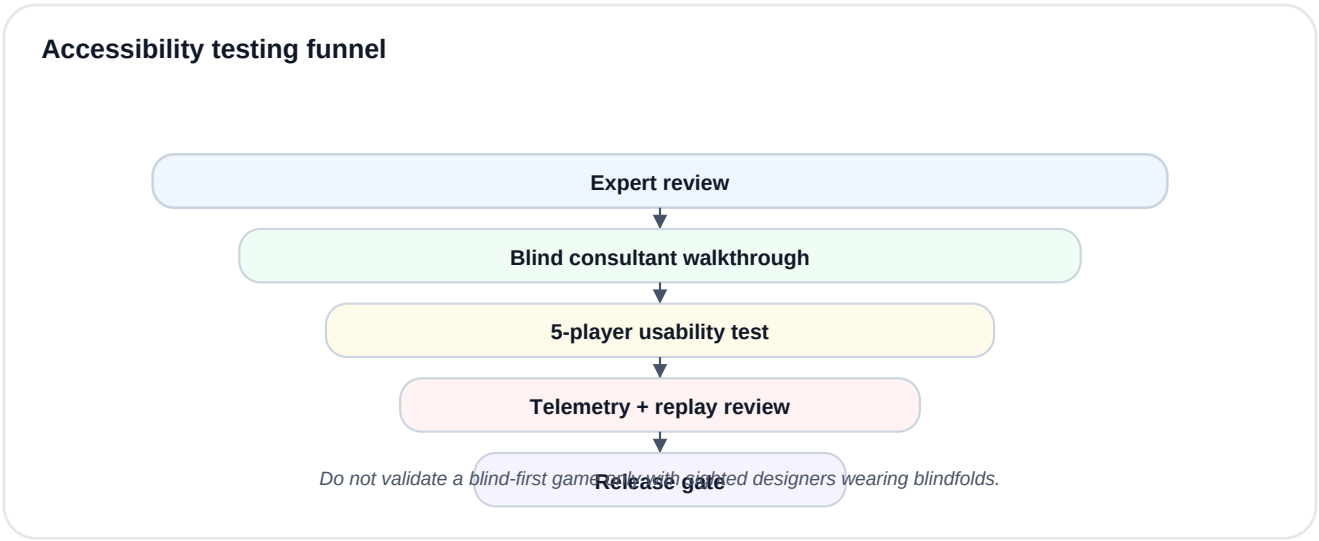
9.5 Visual UI for low-vision players

Although the game is sound-first, low-vision players may benefit from high-contrast visuals, large text, scalable UI, reduced motion, and simple map outlines. Visual UI must never be required, but it can broaden access.

10. Accessibility Testing and Research Protocol

10.1 Testing philosophy

A blind-first game must be tested with blind and low-vision players. Sighted blindfold testing can catch some audio clarity problems, but it cannot replace lived experience. Test with consent, explain recording, allow withdrawal, and compensate participants when possible.



Move from expert review to blind-player testing, telemetry review, and release gates.

10.2 What to measure

Metric	Why it matters	How to collect
First-turn success	Whether movement cues are understandable.	Tutorial telemetry.
Junction decision time	Cognitive load at choices.	Time between junction arrival and direction input.
Avoidable deaths	Fairness of enemy audio.	Replay review and player interview.
Cue confusion	Which sounds are ambiguous.	Post-level questions and replay markers.
Fatigue	Whether audio becomes tiring.	Session duration, self-report, pause frequency.
Learning curve	Whether players improve.	Score, level clear, deaths over repeated runs.

10.3 Test tasks

- Task 1: navigate a straight corridor and stop at a wall.
- Task 2: choose the route with more pellets at a junction.
- Task 3: identify enemy direction and escape.
- Task 4: use power mode and chase an enemy.
- Task 5: clear a small level without trainer help.
- Task 6: change audio settings using only keyboard.

10.4 Release gates

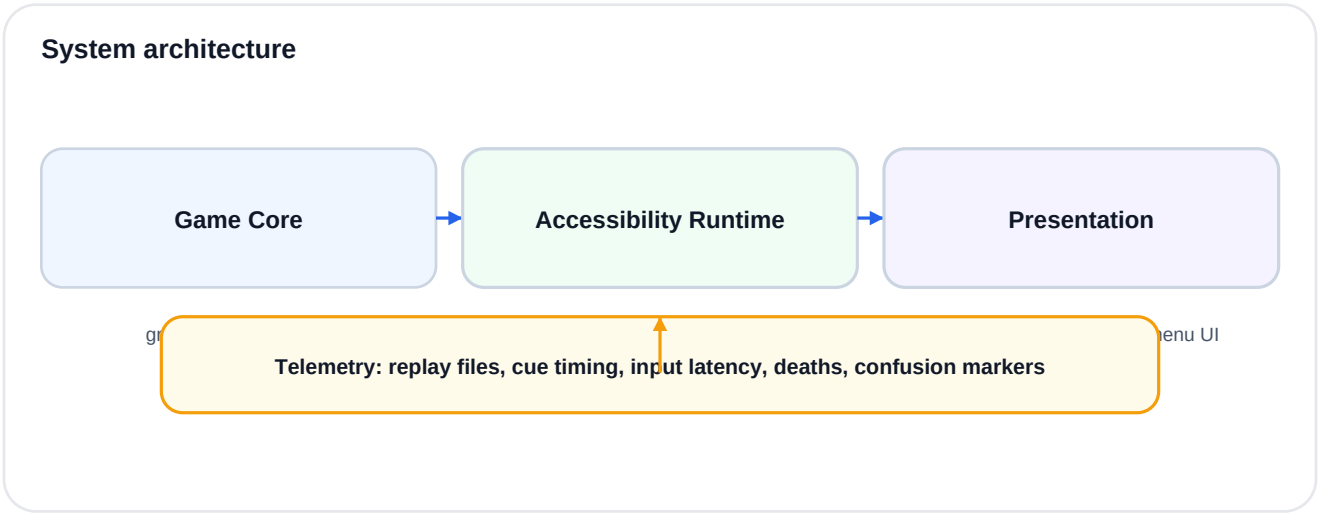
- A new player can start the tutorial without sighted help.
- A player can explain at least 80 percent of live gameplay cue meanings after tutorial.
- Most deaths in early levels are understood by players as fair.
- All menus work with keyboard and screen reader.

- Audio settings are accessible before gameplay starts.
- The game has replay logs for debugging accessibility issues.

11. Production Blueprint and Technical Architecture

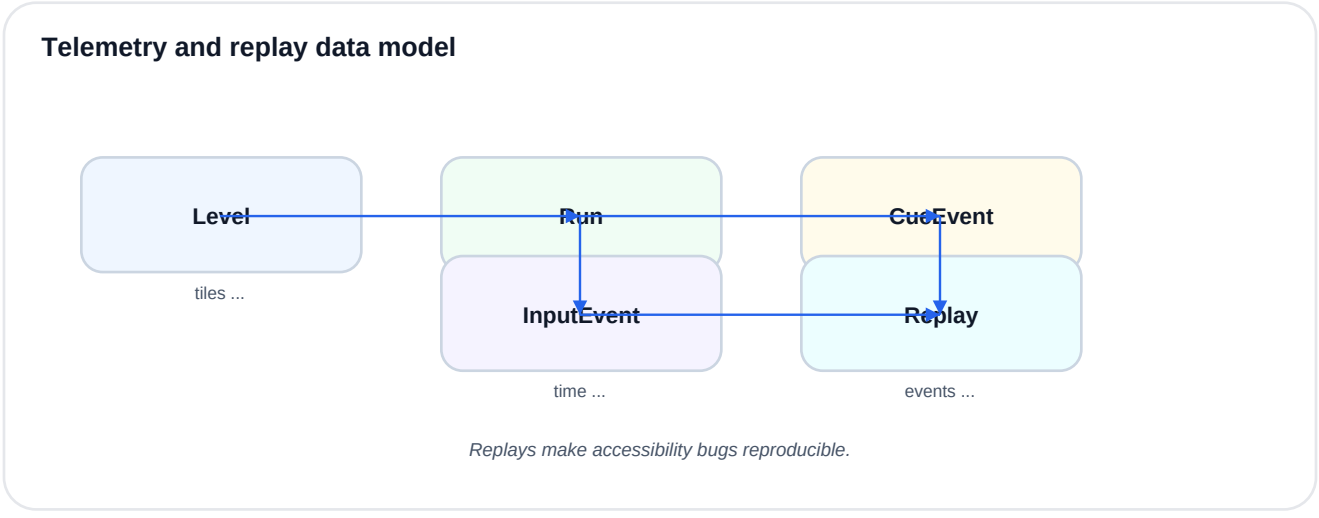
11.1 Recommended architecture

Separate the game core from the accessibility runtime. The game core should know the maze, rules, collisions, scoring, enemy AI, and simulation timing. The accessibility runtime should translate game state into audio cues, speech, input remapping, and trainer state. This separation keeps the game testable and prevents audio from becoming hard-coded chaos.



The game core, accessibility runtime, presentation, and telemetry should be modular.

11.2 Data model



Input events, cue events, level data, runs, and replays should be stored for testing.

11.3 Build milestones

Milestone	Deliverable	Acceptance test
M1 Core movement	Grid movement, collisions, pellets.	Player can clear a visual debug maze.
M2 Audio movement	Pellet, wall, corridor, junction cues.	Player can navigate without looking in a training map.
M3 Enemy AI	One hunter enemy and warning cue.	Player can hear and avoid enemy.
M4 Power mode	Power item, enemy reversal, countdown.	Player can use power strategically.
M5 Menus/tutorial	Keyboard-only onboarding.	New player can start and complete tutorial.
M6 Trainer UI	Visual mirror and cue history.	Trainer can understand player state and audio cues.
M7 Testing pipeline	Replay logs and metrics.	Accessibility bugs can be reproduced from telemetry.

11.4 Audio implementation rules

- Use an event-based audio system with priorities and cooldowns.
- Every cue event should include type, priority, location, duration, and interrupt behavior.
- Danger cues can interrupt reward cues; reward cues should not interrupt danger cues.
- Speech should be queued but cancellable.
- All cues should have IDs so testers can report confusing sounds precisely.
- Telemetry should record both input and audio events for replay.

12. Appendices

Appendix A - Cue lexicon starter kit

Game event	Cue	Priority	Notes
Move one tile	Soft step or pellet tick	Low	Can be reduced in expert mode.
Wall hit	Dry knock	Medium	Short and non-punishing.
Open junction	Three-note directional chime	Medium	Optional in expert mode.
Enemy near	Panned pulse	High	Pulse rate follows time-to-collision.
Power item nearby	Warm resonant hum	Medium	Should be directional.
Power active	Music/filter shift	High	Mode must be obvious.
Power expiring	Countdown tick	High	Never expire silently.
Fruit appears	Short melody	Low/medium	Should lure, not overwhelm.
Life lost	Low cadence + explanation	Critical	In tutorial, explain cause.
Level clear	Celebration cadence	Medium	Then speak score summary.

Appendix B - Level design checklist

- Can the level be completed with visuals off?
- Does every junction have a clear audio identity?
- Are dead ends introduced only after warnings are taught?
- Can a new player recover from confusion?
- Does power mode have activation, countdown, and expiration cues?
- Does each enemy sound distinct enough?
- Does the trainer view explain what the player is hearing?
- Is there a replay log for deaths and confusion?
- Does difficulty increase one variable at a time?
- Can all settings be changed without a mouse?

Appendix C - Failure modes and fixes

Failure mode	Likely cause	Fix
Player says "I do not know where I am."	No landmarks or scan too verbose.	Add zone audio and compact scan summaries.
Player dies without understanding why.	Enemy cue too late or masked.	Increase threat priority and time-to-collision warning.
Player ignores pellets.	Reward cue too weak or too repetitive.	Add progress pitch/rhythm variation.
Player waits too much at junctions.	Choices are unclear.	Add directional route preview and safer early levels.
Audio feels exhausting.	Too many continuous cues.	Use event cues, duck ambience, add cue density settings.
Trainer gives too many answers.	Trainer UI lacks coaching guidance.	Add cue history and prompt: ask before telling.

Appendix D - Bibliography and source notes

This design book is informed by the maze-chase genre, public Pac-Man analysis, and game accessibility research. Important sources include: the official PAC-MAN site for ownership/trademark context; Jamey Pittman's Pac-Man Dossier for classic ghost-mode analysis; Game Accessibility Guidelines for practical accessibility patterns; AbleGamers Accessible Player Experiences for disability-centered game design framing; research on audio-haptic videogaming and spatial navigation for blind users; NavStick research on non-visual virtual environment surveying; and recent reviews of game accessibility for visually impaired players.

Use these sources as background. The actual product should be built through participatory design with blind and low-vision players, not only by reading documents.

Final design commandment

Do not make a blind version of a sighted game. Make a great sound game that sighted people can also play.